UNIDAD 7

Trabajo práctico 07 - Persistencia inicial de reservas de salas

1. Información y entrega

- **Modalidad:** individual.
- **Dominio obligatorio:** reservas de salas de estudio.
- **Punto de partida:** copia funcional del TP 06 de la misma persona.
- **Entrega:** URL de un repositorio individual de GitHub.
- **Vencimiento:** lunes siguiente a las 19:59, antes de la clase de las 20:00.
- **Resultado principal:** crear y recuperar reservas desde MongoDB mediante Mongoose.
- **Entorno de base de datos:** MongoDB Atlas, clúster gratuito Free (M0), igual que en las clases.

Nombrar la copia `tp-07-salas-mongodb` . Conservar intacta la entrega anterior. No copiar `node_modules` , `.git` ni credenciales. Si el TP 06 no funciona, recuperar primero una línea de base ejecutable y documentar la reparación.

2. Consigna

Agregar a la copia un laboratorio de persistencia compuesto por configuración, conexión, modelo y programas separados de creación y consulta. Conservar sin cambios funcionales la aplicación Express del TP 06.

Las rutas web aún operan en memoria. Las reservas persistentes se crean y consultan desde programas Node.js de terminal. El README debe explicar esta separación: no se pide migrar todavía el servicio HTTP ni modificar los controladores para usar promesas.

No entregar una copia literal del ejemplo de productos con nombres cambiados a medias. El modelo, los mensajes y los datos deben corresponder al dominio de reservas.

3. Preparación y estructura

Instalar Mongoose en la copia siguiendo la clase 1. Utilizar el clúster gratuito Atlas Free preparado durante las clases y una base propia llamada `reservas_salas_u7` . Conservar las bases de productos y caminatas: puede reutilizarse el mismo clúster cambiando el nombre de base en la URI privada.

El README debe explicar cómo preparar un clúster Free, un usuario de base con lectura y escritura, la IP permitida y la URI de conexión para Node.js. La instalación local del anexo es material complementario; la evidencia de este TP se obtiene con Atlas.

Conservar el comportamiento de `src/index.js` , `src/app.js` , rutas, controladores, servicio en memoria, middleware, vistas, JSON y recursos previos. Trasladar la función de configuración HTTP a `src/config/entorno.js` , agregar allí el lector de URI y actualizar sus importaciones (incluida la de `src/index.js`) antes de eliminar `src/configuracion.js` . No exigir la URI para arrancar el sitio en memoria. Incorporar archivos con responsabilidades equivalentes a:

```
tp-07-salas-mongodb/
  src/
    config/
      entorno.js
      base-datos.js
    modelos/
      reserva.js
    ...archivos previos del TP 06...
  scripts/
    comprobar-conexion.js
    crear-reserva.js
    listar-reservas.js
  .env.example
  .gitignore
  eslint.config.js
  package.json
  package-lock.json
  README.md
```

Se aceptan otros nombres si el README permite encontrar cada responsabilidad y ejecutar cada comando. No se exigen archivos vacíos para reproducir el árbol.

4. Configuración y conexión

Reutilizar la conexión corregida presentada en clase: importar `node:dns` y configurar `dns.setServers(["8.8.8.8", "1.1.1.1"])` en `src/config/base-datos.js` antes de conectar. No duplicar esta configuración en cada script ni incorporarla al modelo. El apunte de la clase 1 explica SRV/TXT y el alcance del cambio: afecta resolución DNS de Node, no el sistema operativo, la autenticación ni la lista de IPs. Utilizar resolutores autorizados; si la red exige otros o funciona con los predeterminados, documentar el ajuste acordado con el docente. No se evalúa que una red bloquee DNS públicos, sino el diagnóstico y la evidencia real de conexión.

La URI se lee desde `MONGODB_URI` ; no se escribe dentro del modelo ni de los programas de creación/consulta. Si falta o su prefijo es incorrecto, indicar un fallo y terminar sin éxito. No imprimir credenciales ni compartir mensajes completos sin retirar información sensible. Distinguir nombre del error, código DNS reconocido y operación de resolución, como en el programa de prueba de la clase 1.

La plantilla pública `.env.example` conserva PORT y NODE_ENV y agrega una URI de Atlas con marcadores, sin secretos:

```
PORT=3000
NODE_ENV=development
MONGODB_URI=mongodb+srv://USUARIO:CLAVE@HOST/reservas_salas_u7?retryWrites=true&w=majority
```

Copiarla a `.env` y reemplazar allí los marcadores por los datos reales, conservando las opciones de la URI ofrecida por Atlas. `.env` queda excluido de Git; `.env.example` se versiona. Mantener `node_modules` excluido y entregar `package-lock.json` .

La prueba de conexión debe:

1. Esperar la conexión antes de anunciar éxito.
2. Finalizar sin crear reservas.
3. Cerrar los recursos tanto en éxito como en error.
4. Retornar un código de salida distinto de cero si falla.
5. Permitir diagnosticar configuración, servidor y acceso sin publicar la URI privada.

5. Modelo y reglas mínimas

Definir un modelo `Reserva` asociado explícitamente a la colección `reservas` . Trabajar con datos ficticios.

Campo	Tipo en este TP	Regla mínima
estudiante	String	Obligatorio y sin espacios en extremos
email	String	Obligatorio, trim y presencia de @
sala	String	Sala Norte, Sala Sur o Sala Multimedia
fecha	String	Obligatorio y no vacío tras trim
turno	String	Mañana, Tarde o Noche
personas	Number	Obligatorio, entero entre 1 y 6

Sala y turno también son obligatorios. Usar `enum` para opciones, `min` y `max` para límites, y `validate` con `Number.isInteger` para la condición de entero. La presencia de @ puede comprobarse con `match: /@/` , como se presentó el lunes.

Estas reglas mantienen el alcance inicial del curso: no resuelven validación completa de email, fechas imposibles, solapamientos, cupos compartidos ni reservas repetidas. No se exige índice único ni regla de fecha futura.

No generar IDs consecutivos con `reduce` . Usar el `_id` ObjectId y convertirlo a texto solo al mostrarlo. Explicar por qué un _id asignado a un objeto todavía no demuestra una escritura.

Ejecutar desde la raíz: `node --env-file=.env scripts/comprobar-conexion.js` .

6. Programa de creación

Debe conectar, crear **una reserva válida por ejecución** mediante el modelo, esperar el resultado y mostrar el _id de la escritura confirmada. Puede usar un objeto ficticio fijo para esta práctica; no se exige un formulario ni procesar argumentos de terminal.

La confirmación se imprime después de guardar. Un rechazo debe mostrar el tipo de problema y, cuando corresponda, los campos a revisar; no debe presentarse como éxito. Cerrar la conexión en `finally` .

Ejecutar dos veces la creación puede generar dos documentos distintos aunque tengan los mismos campos. Documentar esta conducta; no ocultarla borrando registros.

Ejecutar desde la raíz: `node --env-file=.env scripts/crear-reserva.js` .

7. Programa de consulta

Debe conectar, consultar todas las reservas con el modelo y mostrar un listado legible con _id y campos que permitan reconocerlas, más la cantidad recuperada. Una colección vacía debe producir un resultado válido con cantidad 0.

El listado **no crea ni borra documentos**, no carga automáticamente una semilla y no consulta el JSON del sitio. Debe esperar el resultado de `find` y cerrar la conexión al terminar. No se exige un orden específico.

Ejecutar desde la raíz: `node --env-file=.env scripts/listar-reservas.js` .

8. Evidencia de persistencia

Registrar un ensayo reproducible en la misma base individual:

1. Listar y anotar la cantidad inicial.
2. Crear una reserva y guardar el _id que devuelve.

3. Dejar terminar el programa; iniciar otro proceso de Node.js.
4. Listar y comprobar que se recupera ese mismo _id y que la cantidad creció en uno.
5. Listar una vez más; sin otras escrituras, la cantidad debe permanecer igual.

La evidencia debe permitir distinguir las ejecuciones. Puede ser una transcripción de terminal o capturas recortadas sin secretos. Añadir una captura de Data Explorer de Atlas con la base `reservas_salas_u7` , colección `reservas` y el documento identificado por ese mismo _id. Mostrar solo un mensaje de creación o solo un objeto con _id no alcanza.

9. Matriz mínima de pruebas

Completar resultado observado y cantidad antes/después, sin inventar resultados.

Caso	Resultado esperado
Conexión válida	Confirmación y proceso terminado
Fallo DNS, si ocurrió	Registrar código/operación y solución autorizada; repetir conexión
Variable vacía	Fallo de configuración; sin mensaje de éxito
Contraseña incorrecta temporal en el .env privado	Fallo de autenticación; sin crear documentos
Colección vacía, si se dispone de una base nueva	Listado de cantidad 0, sin error
Reserva válida	Se guarda y devuelve _id
Consulta desde proceso nuevo	Recupera ese _id
Segunda consulta	No agrega registros
Estudiante vacío	Rechazo, cantidad sin cambios
Email sin @	Rechazo, cantidad sin cambios
Sala o turno no permitido	Rechazo, cantidad sin cambios
Fecha vacía	Rechazo, cantidad sin cambios
Personas 0, 7 y 2.5, en pruebas separadas	Rechazo, cantidad sin cambios
Personas con texto no numérico	Rechazo de conversión, sin escritura

Para las pruebas inválidas, modificar temporalmente el objeto del programa de creación de a un campo y restaurarlo al terminar. No eliminar datos para conseguir una colección vacía. Si ya tenía registros, documentar esa condición y probar el vacío en otra base de laboratorio autorizada, si está disponible.

Para probar un fallo de autenticación, conservar la URI válida de manera privada y sustituir temporalmente la contraseña en el `.env` de la copia por un valor incorrecto, sin caracteres reservados. Ejecutar la prueba de conexión: debe fallar sin anunciar éxito. Restaurar la URI original y comprobar la conexión válida. Si la variable está definida también en la terminal, revisar su prioridad antes del ensayo. No cambiar la contraseña del usuario en Atlas para esta prueba. Una variable vacía demuestra configuración ausente; una contraseña incorrecta demuestra autenticación rechazada.

10. Calidad y continuidad del sitio

Ampliar los scripts de calidad del manifiesto a `prettier --write src scripts` , `prettier --check src scripts` y `eslint src scripts` . En `eslint.config.js` , incluir `files: ["src/**/*.js", "scripts/**/*.js"]` , conservando las demás reglas. Ejecutar `npm run format` y `npm run check` sobre ambas carpetas. Los programas de consola importan `../src/config/base-datos` y `../src/modelos/reserva` ; dentro de `src/config/base-datos.js` se importa `./entorno` . Mantener CommonJS. No cambiar las reglas de calidad para ocultar errores.

Comprobar además con `npm start` :

- Inicio, `/estado` y listado siguen disponibles.
- Formulario válido conserva redirección y creación **en memoria**.
- Formulario inválido conserva 400 y no crea.
- CSS y 404 siguen funcionando.

No es necesario repetir aquí todas las capturas del TP 06, pero sí registrar la comprobación. Las reservas del laboratorio no tienen por qué aparecer en las vistas todavía.

11. README obligatorio

Incluir:

1. Punto de partida y nombre de la copia.
2. Versiones utilizadas y confirmación del uso de Atlas Free (M0).
3. Pasos desde una descarga nueva: instalar dependencias, preparar el clúster gratuito, usuario e IP permitida, copiar la plantilla de entorno y ejecutar.
4. Diagnóstico DNS y resolutores usados, si hubo ajustes; comandos exactos de conexión, creación, listado, calidad y arranque web.
5. Mapa de archivos y explicación de esquema, modelo y colección.
6. Campos y reglas, incluyendo límites que todavía no se controlan.
7. Matriz de pruebas con resultados y evidencia de persistencia entre procesos.
8. Explicación de por qué el sitio continúa en memoria y qué se abordará en la unidad 8.
9. Confirmación de que no se publican .env, credenciales ni datos personales.

12. Criterios de evaluación

Criterio	Peso
Preparación reproducible, configuración y conexión	20%
Modelo y reglas del dominio	25%
Creación y consulta correctamente esperadas	20%
Evidencia entre procesos y pruebas negativas	20%
Organización, calidad, README y continuidad HTTP	15%

Una aplicación que solo modifica un arreglo no cumple el resultado principal. Una captura sin pasos reproducibles tampoco demuestra por sí sola la persistencia. Si se expusieron credenciales, corregir la exposición y reemplazarlas antes de considerar segura la entrega.

13. Lista antes de entregar

- La URL corresponde al repositorio correcto y permite al docente acceder.
- Están código, manifiesto, lockfile, plantilla y README.
- No se publicaron node_modules, .env ni credenciales.
- Los programas válidos terminan y los fallidos no anuncian éxito.
- La consulta recupera un _id creado por una ejecución anterior.
- Los casos inválidos no incrementan la cantidad.
- La creación quedó restaurada a un ejemplo válido.
- El sitio previo y los controles de calidad siguen funcionando.